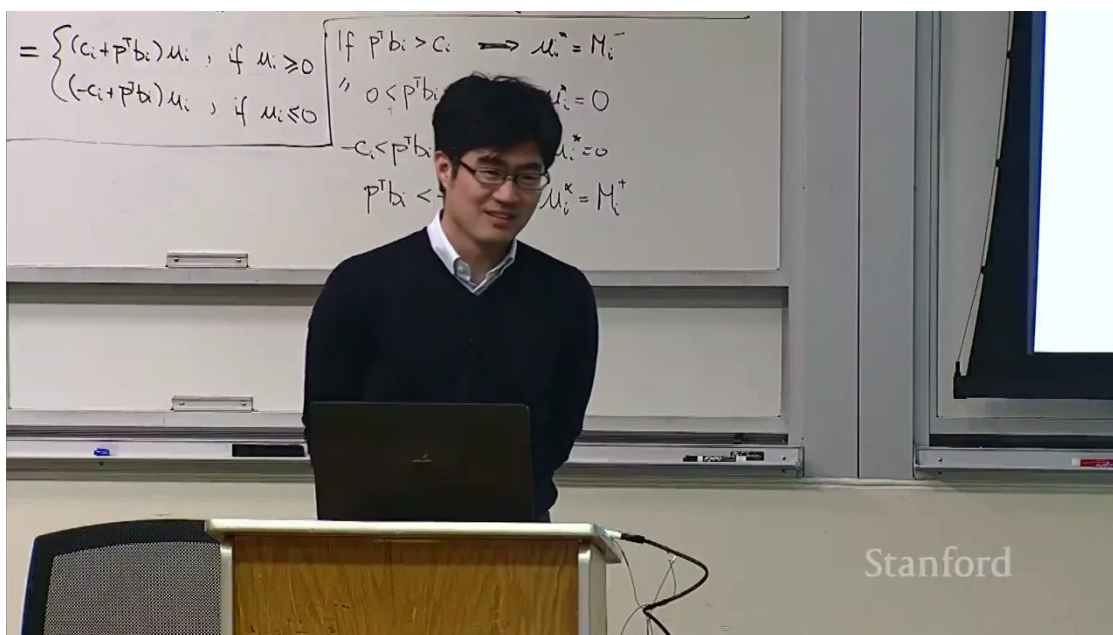


Stanford CS336: Language Modeling from Scratch

Lecture 5: GPUs and TPUs

Spring 2026 中文精讲笔记

主讲: Tatsunori Hashimoto



频道: Stanford Online

发布日期: 2026-04-20

时长: 1:18:39

视频: <https://www.youtube.com/watch?v=izZba4UA7iY>

官方主页: <https://cs336.stanford.edu/>

目录

1 导读：问题、证据与阅读方法	2
2 加速器为何取代通用 CPU	2
3 GPU 与 TPU 的执行和存储层级	7
4 Roofline 与算术强度	12
5 低精度数值格式	17
6 算子融合与重计算	21
7 合并访存和内存对齐	25
8 矩阵乘分块	29
9 注意力与 FlashAttention	33
10 代码化复现：最小机制	37
11 总结与延伸	38
12 资料来源与审计边界	38

1 导读：问题、证据与阅读方法

本讲主问题

语言模型算子怎样映射到 GPU/TPU，才能把峰值算力、内存带宽和片上存储真正转化为训练吞吐？

硬件性能不是一个 TFLOP/s 数字，而是执行层级、存储层级、数据布局 and 内核融合共同作用的结果。本笔记以课程视频、人工字幕和 Stanford 官方讲义或代码为共同证据源。图像均来自每十五秒抽帧后的人工接触表审查，时间脚注可回到原视频复核；公式区分课程机制关系和本笔记的决策分析框架。学习时建议先读主问题，再沿“机制-资源-失败模式-验证指标”四列整理自己的实验。

首先排除的误读

忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

1.1 关键数字核对

主张	数值或关系	视频时间
FP32 每个数的位宽	32 bit	00:34:00
BF16/FP16 每个数的位宽	16 bit	00:34:00
FP8 每个数的位宽	8 bit	00:36:00
MXFP4 每个数的位宽	4 bit	00:41:00
FlashAttention 的核心中间矩阵	不写回完整 $T \times T$ 注意力矩阵	01:15:00

1.2 本章小结

本章建立证据边界：视频解释提供因果和强调，官方材料提供公式与代码，精选帧提供可核查的视觉状态。后文每个主题都同时回答“为何重要、怎样工作、怎样失败、怎样验证”，避免把课程压缩成术语目录。

2 加速器为何取代通用 CPU

本节路线

本节依次处理：GPU 与 TPU 学习资料和硬件资料入口、Dennard 缩放失效后计算需求与硬件供给的分离、GPU 与 CPU 在控制流和并行吞吐上的结构差异、GPU 寄存器共享内存与高带宽内存层级。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$P = \min(P_{\text{peak}}, I \cdot BW)$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。

- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

$$G_1 = \frac{B_1}{C_1 + \lambda R_1 + \varepsilon}$$

- B_1 是本节方法带来的质量、吞吐或覆盖收益。
- C_1 是新增计算、显存、通信、数据或标注成本。
- R_1 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_1 \leq C_{\max}, \quad R_1 \leq R_{\max}, \quad Q_1 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

2.1 GPU 与 TPU 学习资料和硬件资料入口

核心判断。“GPU 与 TPU 学习资料和硬件资料入口”是“加速器为何取代通用 CPU”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 1 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“GPU 与 TPU 学习资料和硬件资料入口”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

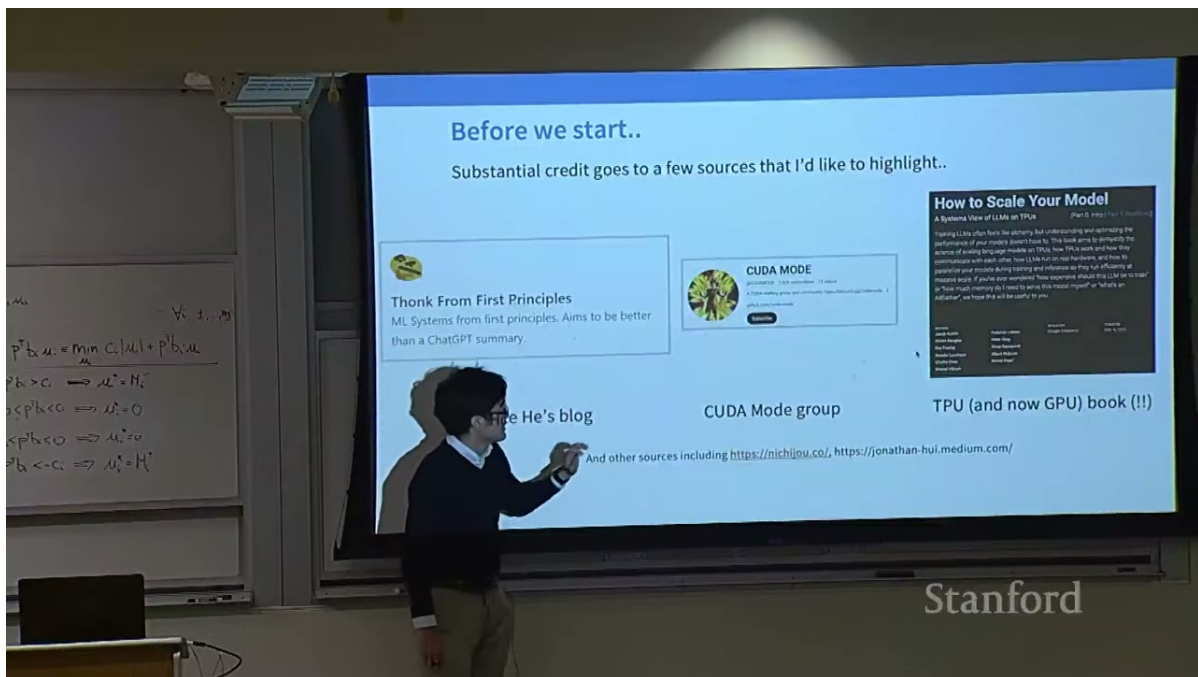


图 1: GPU 与 TPU 学习资料和硬件资料入口¹

2.2 Dennard 缩放失效后计算需求与硬件供给的分离

核心判断。“Dennard 缩放失效后计算需求与硬件供给的分离”是“加速器为何取代通用 CPU”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 2 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“Dennard 缩放失效后计算需求与硬件供给的分离”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹视频画面时间区间：00:02:45–00:03:00。

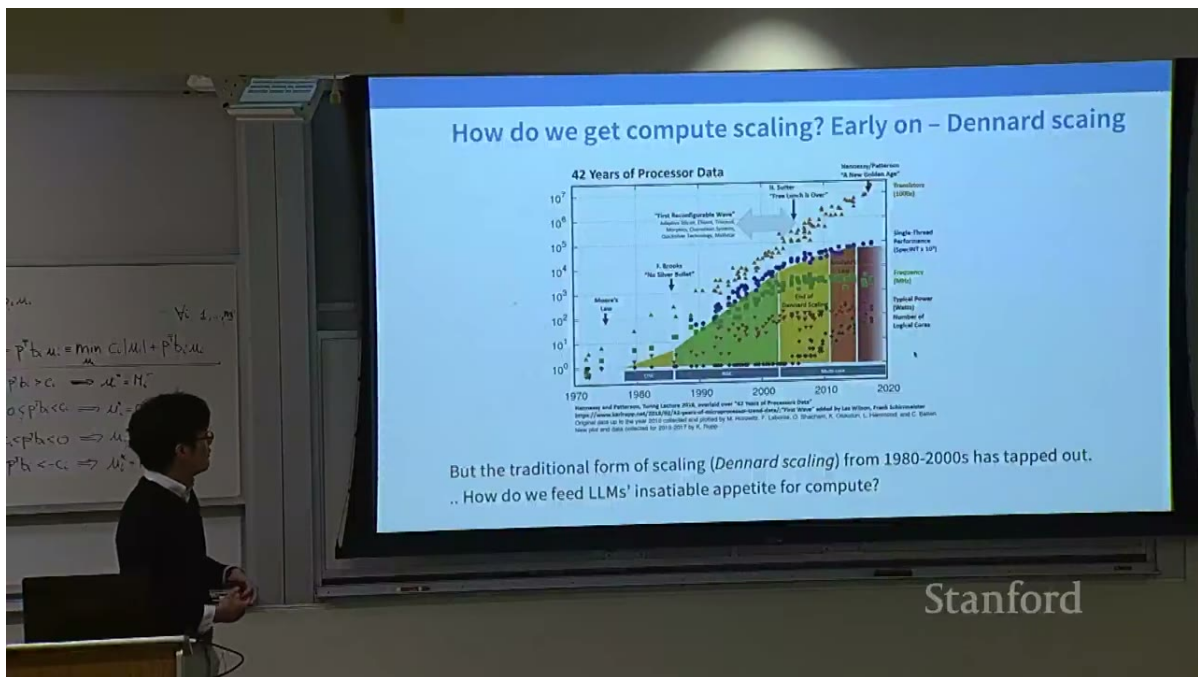


图 2: Dennard 缩放失效后计算需求与硬件供给的分离²

2.3 GPU 与 CPU 在控制流和并行吞吐上的结构差异

核心判断。“GPU 与 CPU 在控制流和并行吞吐上的结构差异”是“加速器为何取代通用 CPU”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 3 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“GPU 与 CPU 在控制流和并行吞吐上的结构差异”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²视频画面时间区间：00:05:30–00:05:45。

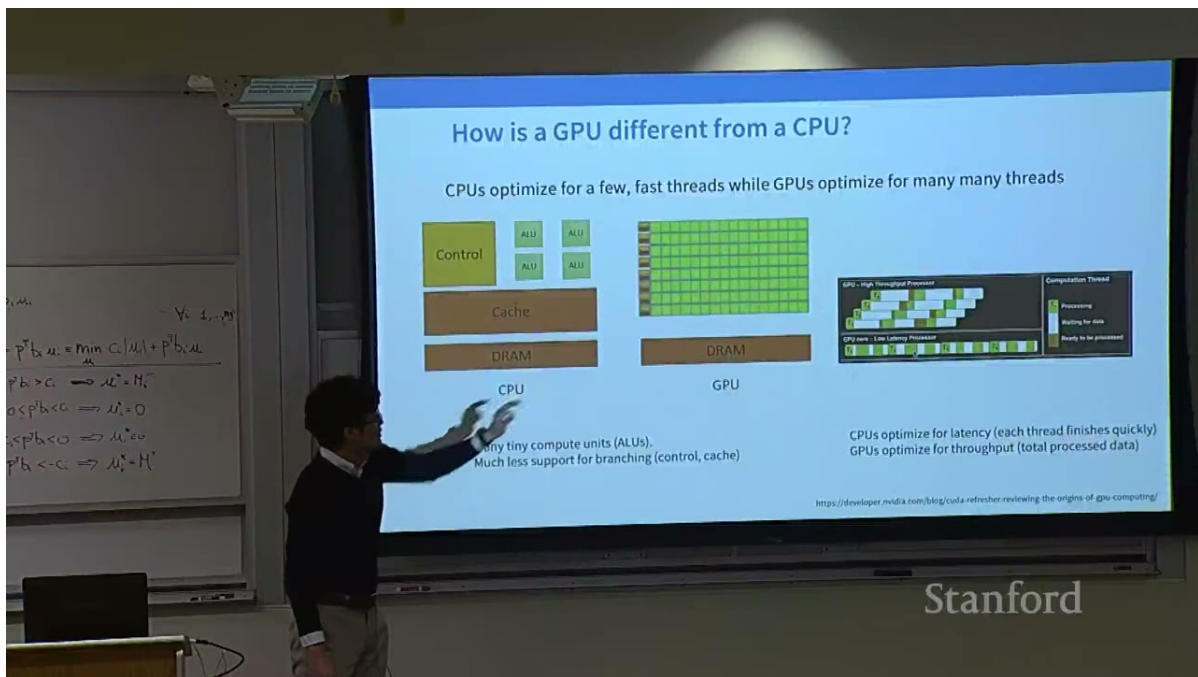


图 3: GPU 与 CPU 在控制流和并行吞吐上的结构差异³

2.4 GPU 寄存器共享内存与高带宽内存层级

核心判断。“GPU 寄存器共享内存与高带宽内存层级”是“加速器为何取代通用 CPU”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 4 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“GPU 寄存器共享内存与高带宽内存层级”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

³视频画面时间区间：00:08:00–00:08:15。

Anatomy of a GPU (memory)

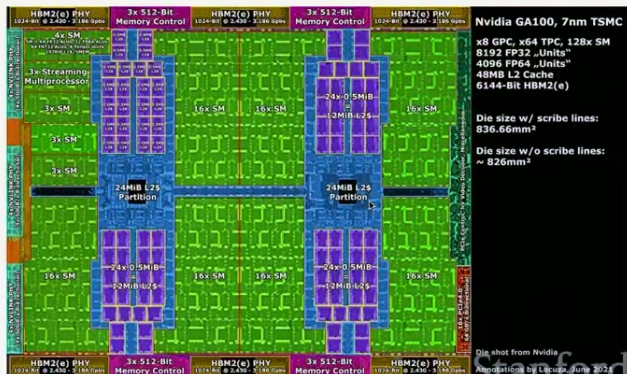
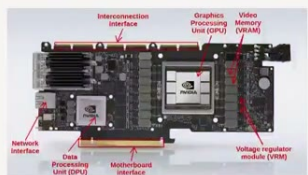
The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

TABLE IV
THE MEMORY ACCESSES LATENCIES



SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

图 4: GPU 寄存器共享内存与高带宽内存层级⁴

2.5 本章小结

“加速器为何取代通用 CPU”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

3 GPU 与 TPU 的执行和存储层级

本节路线

本节依次处理：线程线程块与网格组成的 GPU 执行模型、线程块和流式多处理器之间的数据可见性、TPU 的矩阵乘单元与片上存储结构、GPU 与 TPU 的可编程性和数据流比较。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁴视频画面时间区间：00:10:45-00:11:00。

$$G_2 = \frac{B_2}{C_2 + \lambda R_2 + \epsilon}$$

- B_2 是本节方法带来的质量、吞吐或覆盖收益。
- C_2 是新增计算、显存、通信、数据或标注成本。
- R_2 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_2 \leq C_{\max}, \quad R_2 \leq R_{\max}, \quad Q_2 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

3.1 线程线程块与网格组成的 GPU 执行模型

核心判断。“线程线程块与网格组成的 GPU 执行模型”是“GPU 与 TPU 的执行和存储层级”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 5 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“线程线程块与网格组成的 GPU 执行模型”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

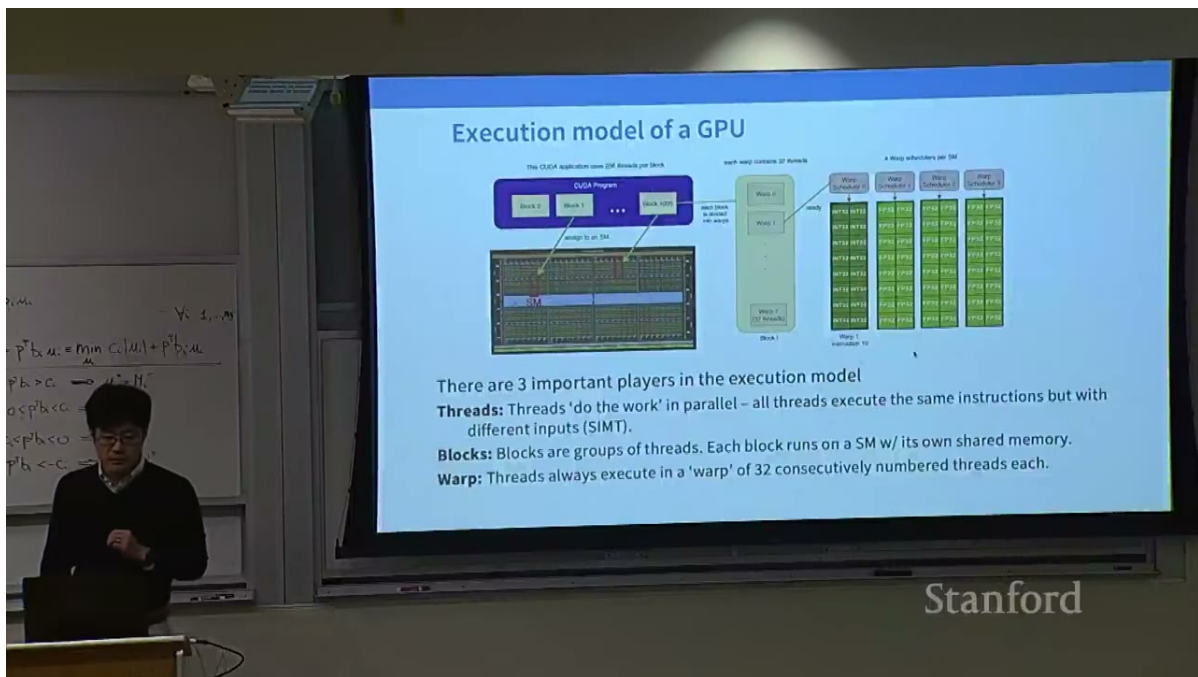


图 5: 线程线程块与网格组成的 GPU 执行模型⁵

3.2 线程块和流式多处理器之间的数据可见性

核心判断。“线程块和流式多处理器之间的数据可见性”是“GPU 与 TPU 的执行和存储层级”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

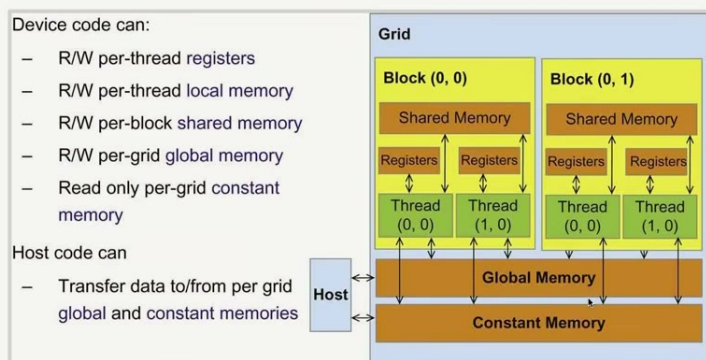
复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 6 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“线程块和流式多处理器之间的数据可见性”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁵视频画面时间区间：00:13:30–00:13:45。

Memory model of a GPU



Each thread can access its own register, and shared memory within the block.

Information that goes across blocks need to be read/written to global memory (slow)

图 6: 线程块和流式多处理器之间的数据可见性⁶

3.3 TPU 的矩阵乘单元与片上存储结构

核心判断。“TPU 的矩阵乘单元与片上存储结构”是“GPU 与 TPU 的执行和存储层级”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 7 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

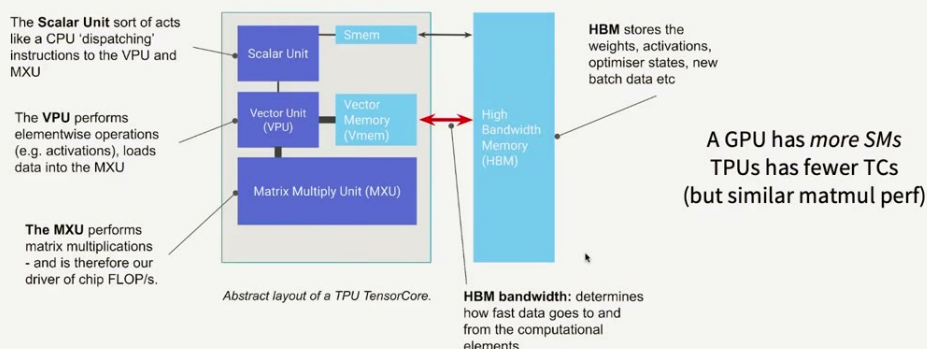
常见失效模式

围绕“TPU 的矩阵乘单元与片上存储结构”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁶视频画面时间区间：00:16:15–00:16:30。

Side thread – What about TPUs?

GPUs, TPUs, and many other accelerators are at a high level, similar



Core structure – lightweight control, fast (big) matmul unit, fast memory.

Differences - how the accelerators are networked (in the parallelism lecture)
- no warps (just blocks – tradeoffs in matmul vs non-matmul)

图 7: TPU 的矩阵乘单元与片上存储结构⁷

3.4 GPU 与 TPU 的可编程性和数据流比较

核心判断。“GPU 与 TPU 的可编程性和数据流比较”是“GPU 与 TPU 的执行和存储层级”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 8 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“GPU 与 TPU 的可编程性和数据流比较”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁷视频画面时间区间：00:19:00–00:19:15。

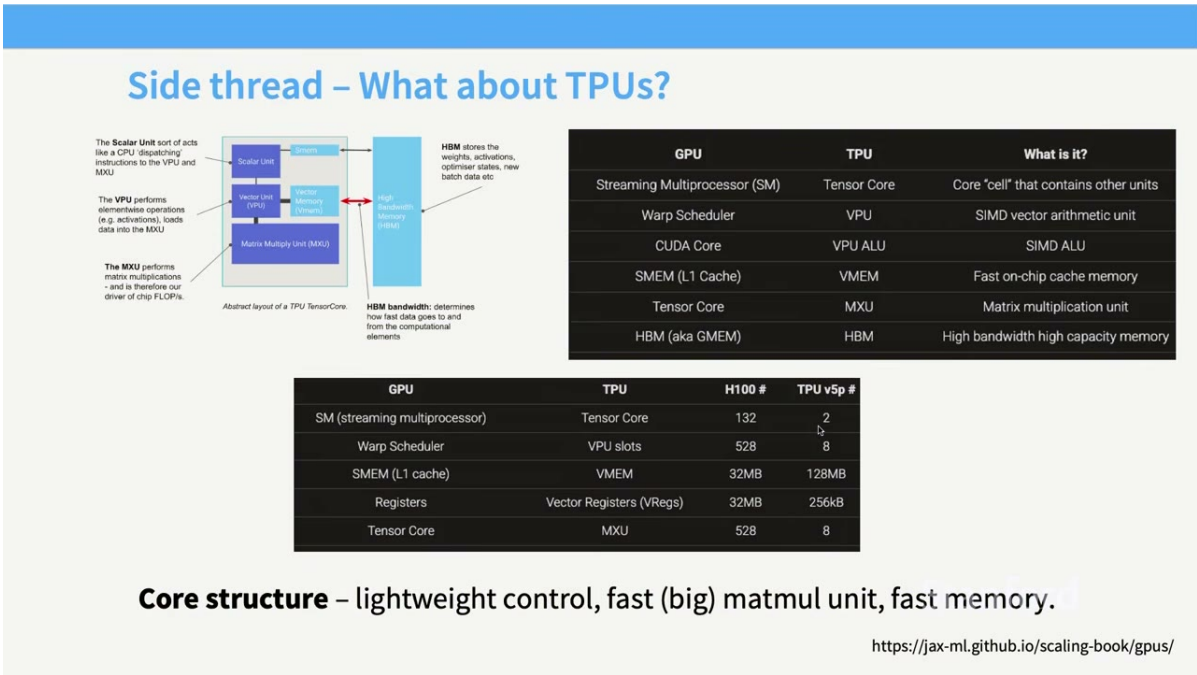


图 8: GPU 与 TPU 的可编程性和数据流比较⁸

3.5 本章小结

“GPU 与 TPU 的执行和存储层级”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

4 Roofline 与算术强度

本节路线

本节依次处理：GPU 矩阵乘吞吐随硬件代际的演进、加速器并行性与存储层级的阶段性总结、计算吞吐和内存带宽共同限定算子性能、低精度融合重计算与访存优化的技术菜单。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$T_{\text{op}} \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{M}{BW}\right)$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁸视频画面时间区间：00:21:45–00:22:00。

$$G_3 = \frac{B_3}{C_3 + \lambda R_3 + \epsilon}$$

- B_3 是本节方法带来的质量、吞吐或覆盖收益。
- C_3 是新增计算、显存、通信、数据或标注成本。
- R_3 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_3 \leq C_{\max}, \quad R_3 \leq R_{\max}, \quad Q_3 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

4.1 GPU 矩阵乘吞吐随硬件代际的演进

核心判断。“GPU 矩阵乘吞吐随硬件代际的演进”是“Roofline 与算术强度”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 9 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“GPU 矩阵乘吞吐随硬件代际的演进”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

GPUs as fast matrix multipliers

Fast Matrix Multiplies using Graphics Hardware

E. Scott Larsen
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
larsene@cs.unc.edu

David McAllister
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
davemc@cs.unc.edu

Implementation

We mention here some observations we made during our implementation that may be of interest to those duplicating our results.

Refresh Rate We found that setting the refresh rate on the monitor as low as possible made marginal improvements (about 10%).

RGBA We found that 4 numbers can be packed into a single pixel, by setting the red, green, blue, and alpha channels to different values.

Texture Format Changing the format of the texture creation and read-back from RGBA to ABGR_EXT (in OpenGL) gave about 40% improvement on our hardware. This is because the hardware driver avoids reformatting the data from the application format to the card format. There is a number of options here, with near equal performance for each option except the one used natively on the specific hardware. The native format should give significant improvement.

Full Screen Running full screen instead of in a window provides improved performance.

Various other optimizations yielded minor (<1%) improvements.

Early days of NVIDIA GPUs – programmable shaders. Researchers hacked this to do matmuls

图 9: GPU 矩阵乘吞吐随硬件代际的演进⁹

4.2 加速器并行性与存储层级的阶段性总结

核心判断。“加速器并行性与存储层级的阶段性总结”是“Roofline 与算术强度”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 10 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“加速器并行性与存储层级的阶段性总结”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁹视频画面时间区间：00:24:15–00:24:30。

Recap: GPUs – what are they and how do they work

- ❖ GPUs are massively parallel – same instructions applied across many workers
- ❖ Compute (and esp matmuls) have scaled faster than memory
- ❖ We have to respect the memory hierarchy to make things go fast.

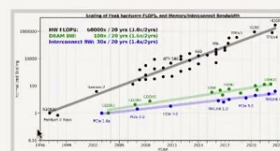


TABLE IV
THE MEMORY ACCESS LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

图 10: 加速器并行性与存储层级的阶段性总结¹⁰

4.3 计算吞吐和内存带宽共同限定算子性能

核心判断。“计算吞吐和内存带宽共同限定算子性能”是“Roofline 与算术强度”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 11 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“计算吞吐和内存带宽共同限定算子性能”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁰视频画面时间区间：00:27:00–00:27:15。

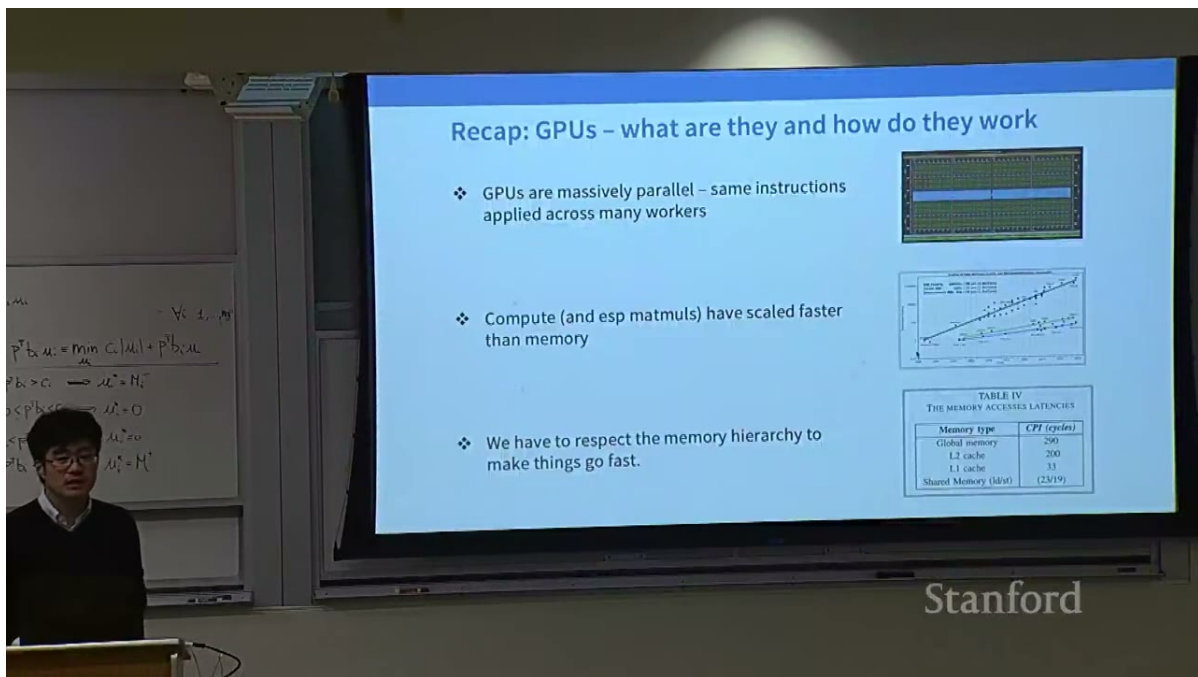


图 11: 计算吞吐和内存带宽共同限定算子性能¹¹

4.4 低精度融合重计算与访存优化的技术菜单

核心判断。“低精度融合重计算与访存优化的技术菜单”是“Roofline 与算术强度”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 12 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“低精度融合重计算与访存优化的技术菜单”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹¹视频画面时间区间：00:29:45–00:30:00。

How do we make GPUs go fast?

1. Control divergence (not a memory bottleneck..)
2. Low precision computation
3. Operator fusion
4. Recomputation
5. Coalescing memory
6. Tiling

图 12: 低精度融合重计算与访存优化的技术菜单¹²

4.5 本章小结

“Roofline 与算术强度”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

5 低精度数值格式

本节路线

本节依次处理：低精度表示通过减少字节数提高有效吞吐、FP8 指数位和尾数位的格式选择、E4M3 与 E5M2 的范围精度权衡。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$M_{\text{number}} = n \frac{b}{8}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹²视频画面时间区间：00:32:30–00:32:45。

$$G_4 = \frac{B_4}{C_4 + \lambda R_4 + \epsilon}$$

- B_4 是本节方法带来的质量、吞吐或覆盖收益。
- C_4 是新增计算、显存、通信、数据或标注成本。
- R_4 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_4 \leq C_{\max}, \quad R_4 \leq R_{\max}, \quad Q_4 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

5.1 低精度表示通过减少字节数提高有效吞吐

核心判断。“低精度表示通过减少字节数提高有效吞吐”是“低精度数值格式”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 13 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“低精度表示通过减少字节数提高有效吞吐”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

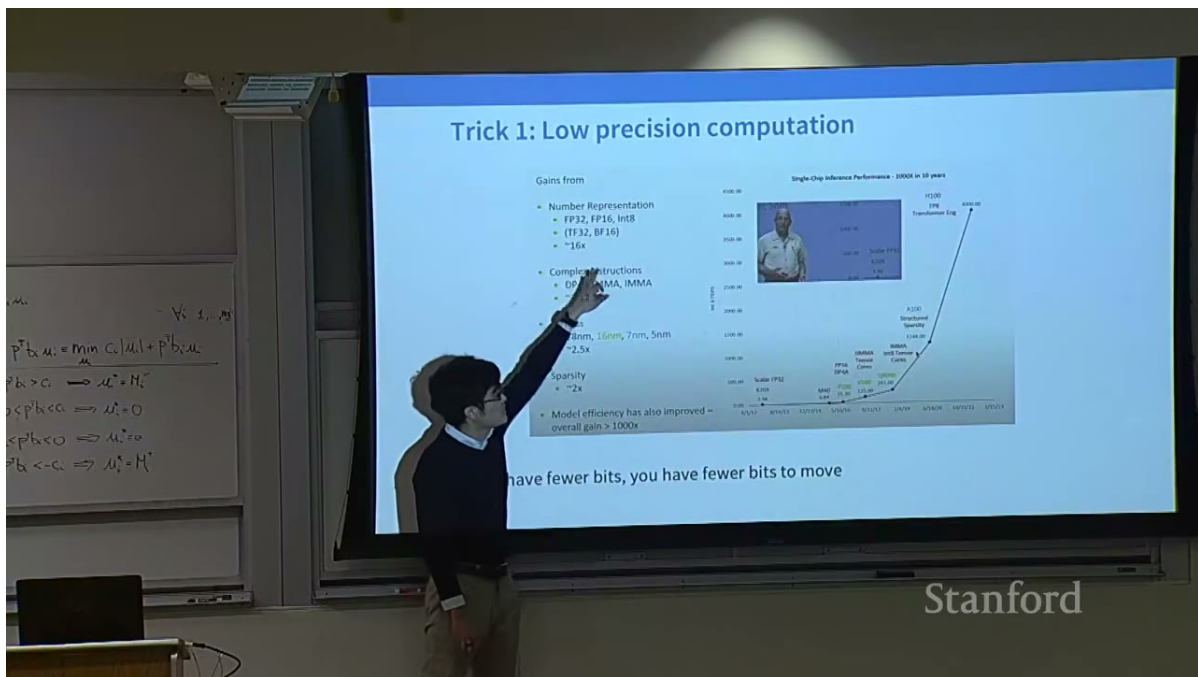


图 13: 低精度表示通过减少字节数提高有效吞吐¹³

5.2 FP8 指数位和尾数位的格式选择

核心判断。“FP8 指数位和尾数位的格式选择”是“低精度数值格式”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 14 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

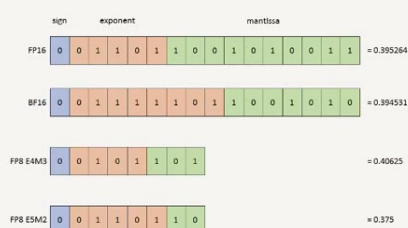
复现实验记录

围绕“FP8 指数位和尾数位的格式选择”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹³视频画面时间区间：00:35:15–00:35:30。

Frontiers in low precision

Very low precision (FP8) with different tradeoffs



MXFP8 has many *interesting* things about it

- Uses E4M3 (more mantissa) due to more scale factors
- Scale factors themselves are FP8 (E8M0), 1 per 32
- Transposes are now nontrivial!

Multiple scaling factors MXFP8 (Blackwell)

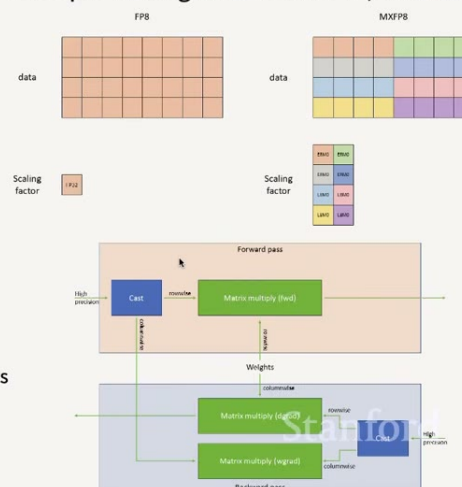


图 14: FP8 指数位和尾数位的格式选择¹⁴

5.3 E4M3 与 E5M2 的范围精度权衡

核心判断。“E4M3 与 E5M2 的范围精度权衡”是“低精度数值格式”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 15 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

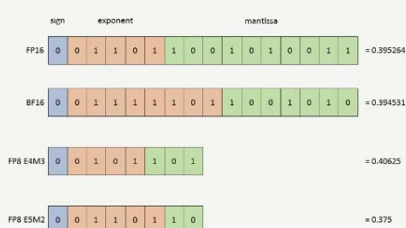
常见失效模式

围绕“E4M3 与 E5M2 的范围精度权衡”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁴视频画面时间区间：00:38:00–00:38:15。

Frontiers in low precision

Very low precision (FP8) with different tradeoffs



MXFP8 has many *interesting* things about it

- Uses E4M3 (more mantissa) due to more scale factors
- Scale factors themselves are FP8 (E8M0), 1 per 32
- Transposes are now nontrivial!

Multiple scaling factors MXFP8 (Blackwell)

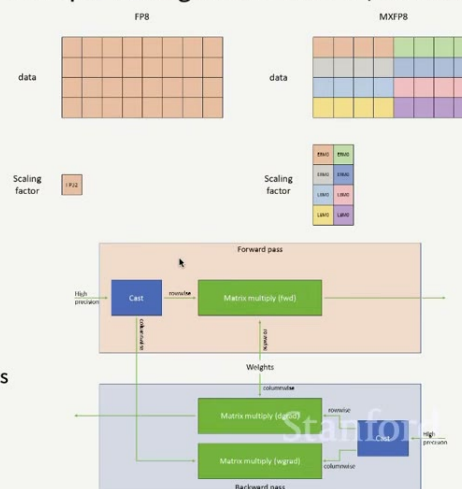


图 15: E4M3 与 E5M2 的范围精度权衡¹⁵

5.4 本章小结

“低精度数值格式”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

6 算子融合与重计算

本节路线

本节依次处理：MXFP4 共享尺度与四位权重表示、正弦余弦计算合并为单个融合算子、以重计算换取更低激活内存占用。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\hat{x} = s \cdot \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁵视频画面时间区间：00:40:30–00:40:45。

$$G_5 = \frac{B_5}{C_5 + \lambda R_5 + \epsilon}$$

- B_5 是本节方法带来的质量、吞吐或覆盖收益。
- C_5 是新增计算、显存、通信、数据或标注成本。
- R_5 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_5 \leq C_{\max}, \quad R_5 \leq R_{\max}, \quad Q_5 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

6.1 MXFP4 共享尺度与四位权重表示

核心判断。“MXFP4 共享尺度与四位权重表示”是“算子融合与重计算”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 16 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“MXFP4 共享尺度与四位权重表示”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

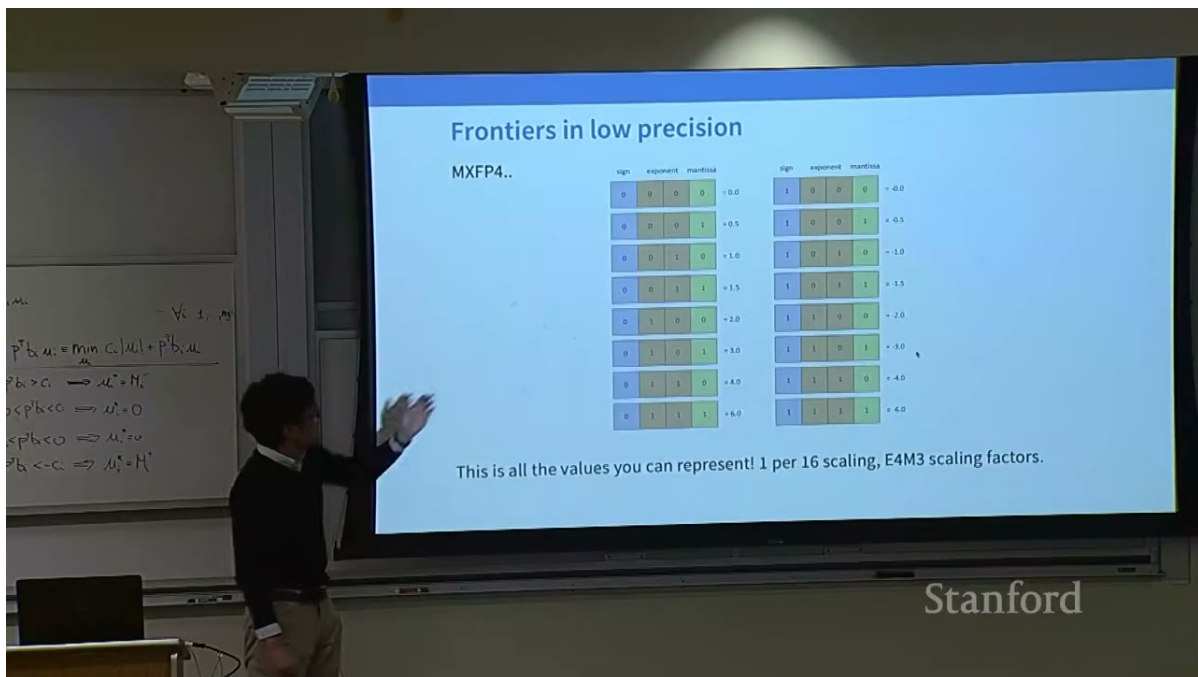


图 16: MXFP4 共享尺度与四位权重表示¹⁶

6.2 正弦余弦计算合并为单个融合算子

核心判断。“正弦余弦计算合并为单个融合算子”是“算子融合与重计算”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

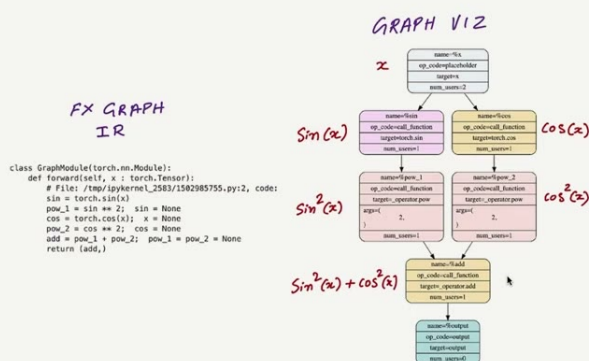
复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 17 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“正弦余弦计算合并为单个融合算子”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁶视频画面时间区间：00:43:15–00:43:30。

Example – sines and cosines



Computing $\sin^2 x + \cos^2 x$ naively launches 5 CUDA kernels (back and forth)

<https://towardsdatascience.com/how-pytorch-2-0-accelerates-deep-learning-with-operator-fusion-and-cpu-gpu-code-generation-35132a85bd26>

图 17: 正弦余弦计算合并为单个融合算子¹⁷

6.3 以重计算换取更低激活内存占用

核心判断。“以重计算换取更低激活内存占用”是“算子融合与重计算”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

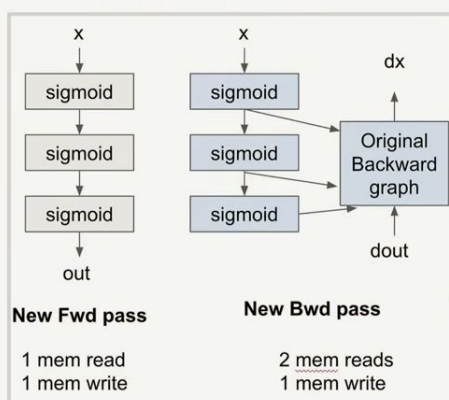
复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 18 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“以重计算换取更低激活内存占用”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁷视频画面时间区间：00:48:45–00:49:00。

Throw away the activations, re-compute them!



Throwing away computation can actually be optimal, w/ 5/8th the memory accesses!

<https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467>

图 18: 以重计算换取更低激活内存占用¹⁸

6.4 本章小结

“算子融合与重计算”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

7 合并访存和内存对齐

本节路线

本节依次处理：连续线程合并访问全局内存、矩阵乘索引布局对合并访存的影响、共享内存分块复用矩阵乘输入。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$N_{\text{transactions}} \geq \left\lceil \frac{M_{\text{requested}}}{M_{\text{transaction}}} \right\rceil$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁸视频画面时间区间：00:51:30–00:51:45。

$$G_6 = \frac{B_6}{C_6 + \lambda R_6 + \varepsilon}$$

- B_6 是本节方法带来的质量、吞吐或覆盖收益。
- C_6 是新增计算、显存、通信、数据或标注成本。
- R_6 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_6 \leq C_{\max}, \quad R_6 \leq R_{\max}, \quad Q_6 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

7.1 连续线程合并访问全局内存

核心判断。“连续线程合并访问全局内存”是“合并访存和内存对齐”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

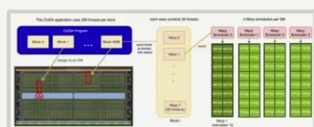
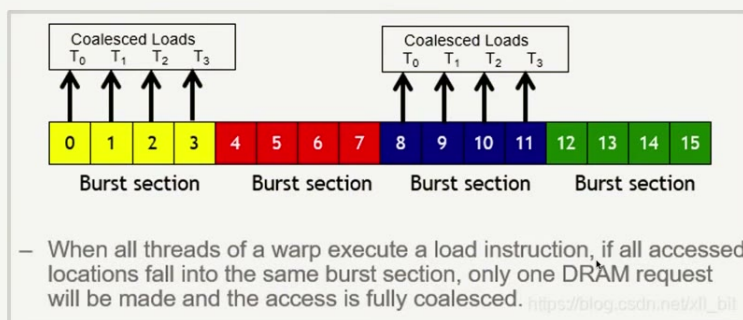
复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 19 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“连续线程合并访问全局内存”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

Memory coalescing

Memory accesses are *coalesced* if all the threads (in a warp) fall within the same burst



Reminder: a warp is a set of 32 consecutively numbered threads that execute together in a block. Memory accesses happen together

图 19: 连续线程合并访问全局内存¹⁹

7.2 矩阵乘索引布局对合并访存的影响

核心判断。“矩阵乘索引布局对合并访存的影响”是“合并访存和内存对齐”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 20 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“矩阵乘索引布局对合并访存的影响”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁹视频画面时间区间：00:54:15–00:54:30。

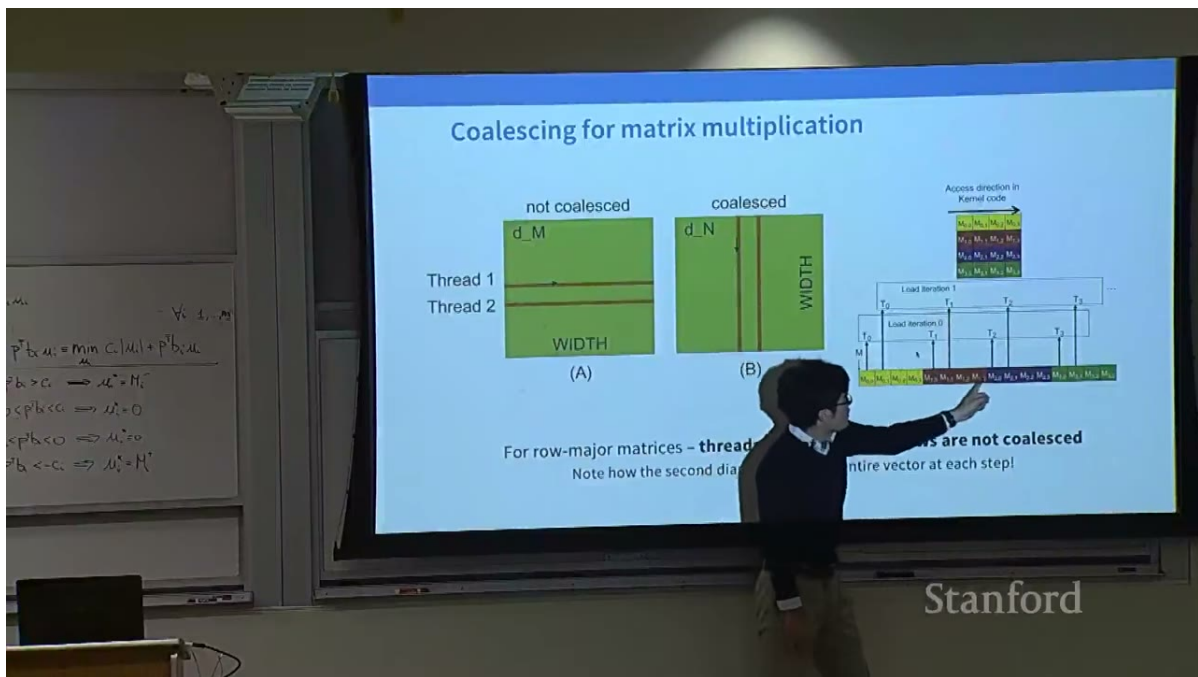


图 20: 矩阵乘索引布局对合并访存的影响²⁰

7.3 共享内存分块复用矩阵乘输入

核心判断。“共享内存分块复用矩阵乘输入”是“合并访存和内存对齐”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 21 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

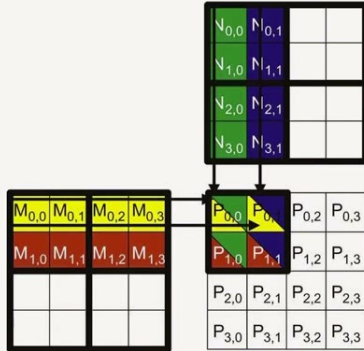
常见失效模式

围绕“共享内存分块复用矩阵乘输入”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁰视频画面时间区间：00:56:45–00:57:00。

Tiling – store and reuse information in shared memory

Cut up the matrix into smaller ‘tiles’, and load this into shared memory



Compute the matrix multiply in ‘phases’

1. Load $M_{0,0}$ and $N_{0,0}$ tiles into SHM
2. Compute partial sums for P
(Done with one tile)
3. Load the $M_{0,0}$ and $N_{2,0}$ tile into SHM
4. ...

Advantages: repeated reads now access shared, not global memory and memory access can be coalesced

图 21: 共享内存分块复用矩阵乘输入²¹

7.4 本章小结

“合并访存和内存对齐”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

8 矩阵乘分块

本节路线

本节依次处理：分块尺寸和尾块浪费决定硬件利用率、内存对齐对向量化加载和事务数量的影响、分块矩阵乘的实测性能对比。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I_{\text{tile}} \approx \frac{2mnk}{b(mk + kn + mn)}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

²¹视频画面时间区间：00:59:30–00:59:45。

$$G_7 = \frac{B_7}{C_7 + \lambda R_7 + \epsilon}$$

- B_7 是本节方法带来的质量、吞吐或覆盖收益。
- C_7 是新增计算、显存、通信、数据或标注成本。
- R_7 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_7 \leq C_{\max}, \quad R_7 \leq R_{\max}, \quad Q_7 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

8.1 分块尺寸和尾块浪费决定硬件利用率

核心判断。“分块尺寸和尾块浪费决定硬件利用率”是“矩阵乘分块”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 22 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“分块尺寸和尾块浪费决定硬件利用率”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

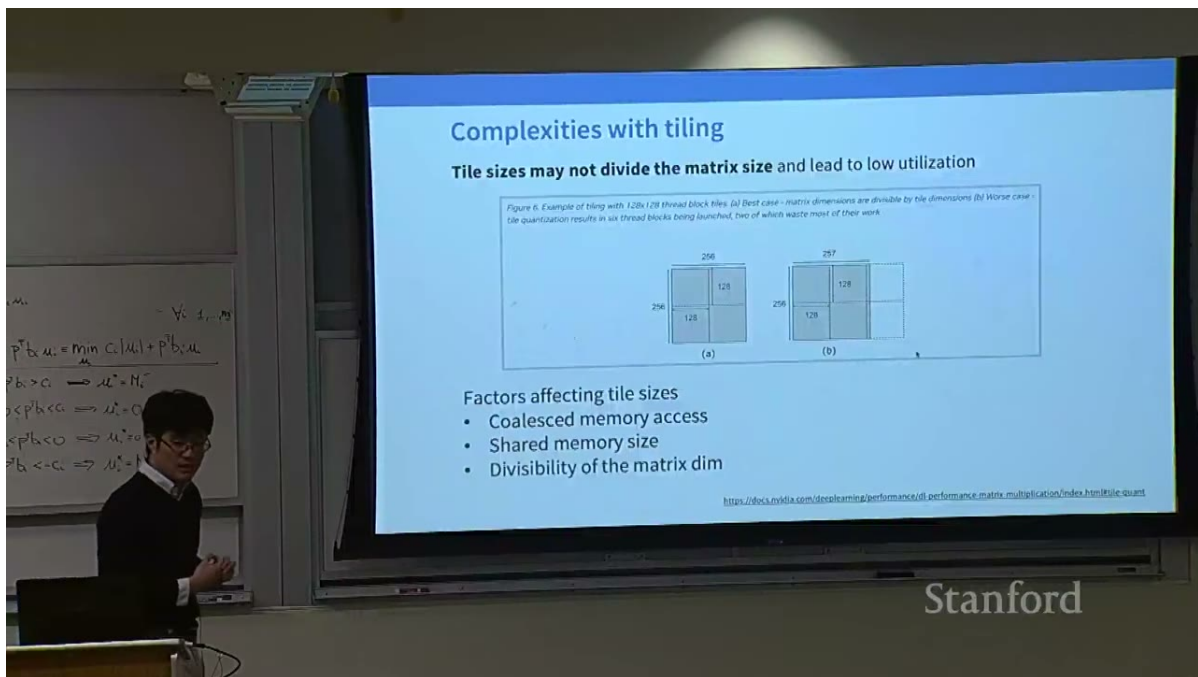


图 22: 分块尺寸和尾块浪费决定硬件利用率²²

8.2 内存对齐对向量化加载和事务数量的影响

核心判断。“内存对齐对向量化加载和事务数量的影响”是“矩阵乘分块”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 23 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“内存对齐对向量化加载和事务数量的影响”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²²视频画面时间区间：01:02:15–01:02:30。

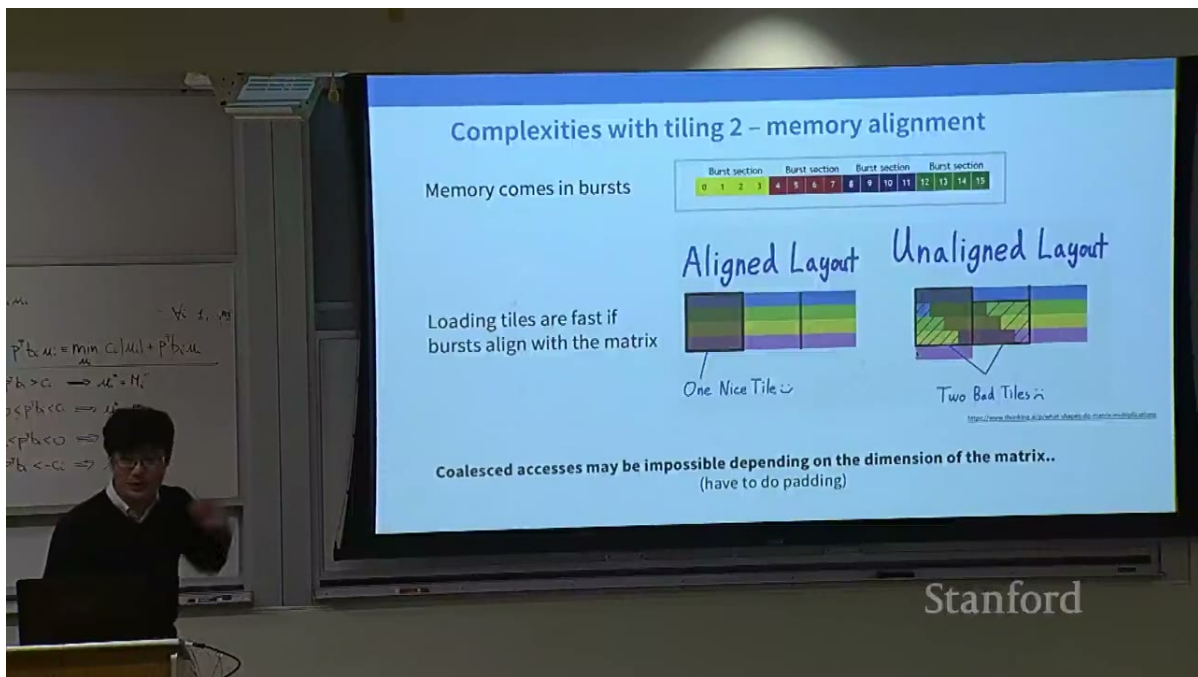


图 23: 内存对齐对向量化加载和事务数量的影响²³

8.3 分块矩阵乘的实测性能对比

核心判断。“分块矩阵乘的实测性能对比”是“矩阵乘分块”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

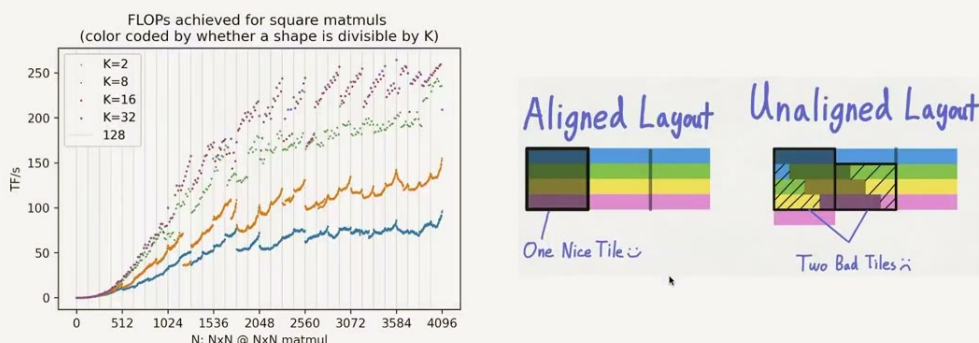
复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 24 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“分块矩阵乘的实测性能对比”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²³视频画面时间区间：01:05:00–01:05:15。

Part 1: tiling



Tiling has a major impact through alignment.

图 24: 分块矩阵乘的实测性能对比²⁴

8.4 本章小结

“矩阵乘分块”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

9 注意力与 FlashAttention

本节路线

本节依次处理：算术强度精度与访存模式的联合检查、注意力计算的张量形状和数据移动、FlashAttention 用分块融合避免写出注意力矩阵。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

²⁴视频画面时间区间：01:07:45–01:08:00。

$$G_8 = \frac{B_8}{C_8 + \lambda R_8 + \epsilon}$$

- B_8 是本节方法带来的质量、吞吐或覆盖收益。
- C_8 是新增计算、显存、通信、数据或标注成本。
- R_8 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_8 \leq C_{\max}, \quad R_8 \leq R_{\max}, \quad Q_8 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

9.1 算术强度精度与访存模式的联合检查

核心判断。“算术强度精度与访存模式的联合检查”是“注意力与 FlashAttention”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 25 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“算术强度精度与访存模式的联合检查”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

Recap of part 2: making ML workloads go fast

❖ Reduce memory accesses

❖ Coalescing

❖ Fusion

❖ Move memory to shared memory

❖ Tiling

❖ Trade memory for compute/accuracy

❖ Quantization

❖ Recomputation

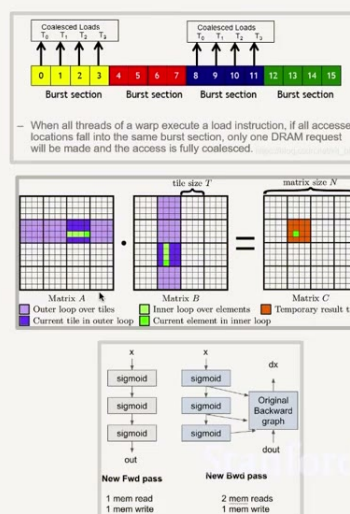


图 25: 算术强度精度与访存模式的联合检查²⁵

9.2 注意力计算的张量形状和数据移动

核心判断。“注意力计算的张量形状和数据移动”是“注意力与 FlashAttention”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 26 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“注意力计算的张量形状和数据移动”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁵视频画面时间区间：01:10:30–01:10:45。

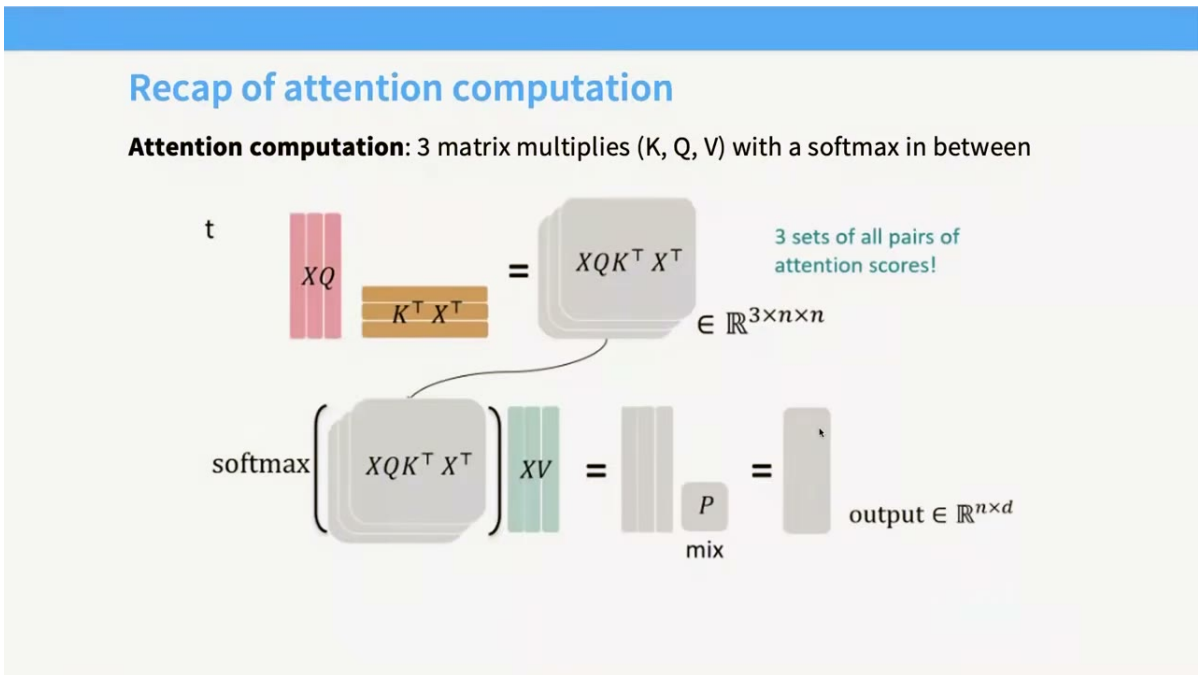


图 26: 注意力计算的张量形状和数据移动²⁶

9.3 FlashAttention 用分块融合避免写出注意力矩阵

核心判断。“FlashAttention 用分块融合避免写出注意力矩阵”是“注意力与 FlashAttention”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 27 个图像证据在本讲 27 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“FlashAttention 用分块融合避免写出注意力矩阵”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁶视频画面时间区间：01:13:00–01:13:15。

Putting it all together – the forward pass of flash attention

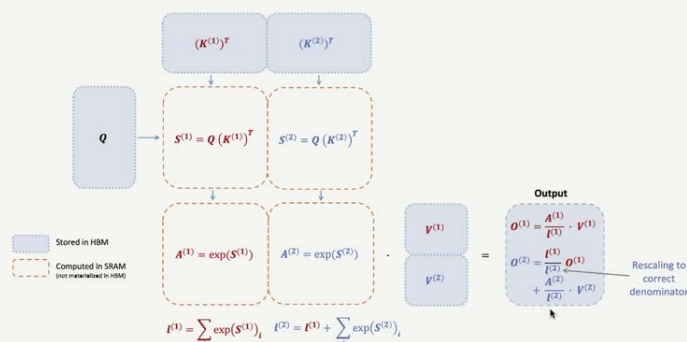


图 27: FlashAttention 用分块融合避免写出注意力矩阵²⁷

9.4 本章小结

“注意力与 FlashAttention” 的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。

10 代码化复现：最小机制

下面的片段保留本讲最核心的程序结构，用于把公式和系统过程落到可执行对象。它不是生产实现；实际复现还需补充张量形状断言、数值精度、同步、随机种子、数据版本和端到端测试。

Listing 1: Lecture 5 机制伪代码

```
1 # 伪代码：把逐元素算子和矩阵乘后的偏置/激活融合
2 y = matmul(x, w)
3 y = fused_bias_gelu(y, bias) # 少一次 HBM 写回与读取
```

复现检查单

先写正确性基线，再记录资源基线；每次只改变一个机制；同时保存直接中间量和最终指标；对随机算法报告多次运行；对数据与评测保存不可变版本标识。

²⁷视频画面时间区间：01:15:45–01:16:00。

10.1 本章小结

代码的价值在于暴露状态、顺序和边界。若一个概念无法写成输入、变换、状态、输出和断言，它通常还没有被真正理解。

11 总结与延伸

一页压缩

本讲的主问题是：语言模型算子怎样映射到 GPU/TPU，才能把峰值算力、内存带宽和片上存储真正转化为训练吞吐？统一答案是：硬件性能不是一个 TFLOP/s 数字，而是执行层级、存储层级、数据布局 and 内核融合共同作用的结果。

第一，优先理解机制而不是产品名。模型、硬件、数据和评测不断更新，但张量形状、状态位置、目标函数、通信边界和测量误差仍然是稳定的分析对象。第二，所有优化都要写出代价守恒：减少某一种成本通常会增加另一种成本或风险。第三，把小规模实验推广到大规模之前，必须检查瓶颈是否改变、数据分布是否改变、实现常数是否仍在同一数量级。第四，最可靠的课程笔记应允许读者沿时间轴注回到原视频，并沿官方材料找到公式或代码出处。

开放问题

忽略访存会把 FLOPs 优化成纸面指标；忽略数值范围又会让低精度带来的吞吐收益变成训练不稳定。后续实验应把这一风险写成可观测量和停止条件，而不是留在讨论部分。

11.1 本章小结

完成本讲后，读者应能用统一语言解释每个主题的输入、状态、变换、输出、资源成本和失败模式，并据此设计一个含基线、消融、误差条和证据记录的最小实验。

12 资料来源与审计边界

- 课程主页: <https://cs336.stanford.edu/>。
- 视频: <https://www.youtube.com/watch?v=izZba4UA7iY>。
- Stanford 官方材料: lecture_05.pdf; 客座讲座若无官方文件则以视频和字幕为主。
- 本地证据: audio.srt, figure_manifest.tsv, figure_verification.txt, teaching_atoms.tsv, numerical_claims.tsv。

12.1 本章小结

本笔记覆盖播放列表中的本讲内容与课程主页当日可取得的官方材料。没有官方讲义的客座讲座不会被伪装成有讲义；讲者口头推测、本笔记分析框架和官方公式在文字中分别标明。